

第2章 神经网络设计实验

神经网络设计是实现复杂深度学习算法/应用的基础，本章将介绍如何设计一个三层神经网络模型来实现手写数字分类。首先介绍在 CPU 平台上如何利用高级编程语言 Python 搭建神经网络训练和推断框架来实现手写数字分类，随后介绍如何将前述算法移植到深度学习处理器 DLP 上。完成本章实验，读者就可以点亮智能计算系统知识树（图1.2）中神经网络设计部分的知识点。

2.1 基于三层神经网络实现手写数字分类

2.1.1 实验目的

掌握神经网络的设计原理，熟练掌握神经网络的训练和推断方法；能够使用 Python 语言实现一个三层全连接神经网络模型对手写数字分类的训练和推断。具体包括：

1) 实现三层神经网络模型进行手写数字分类，建立一个简单而完整的神经网络工程。通过本实验理解神经网络中基本模块的作用和模块间的关系，为后续建立更复杂的神经网络（如风格迁移）奠定基础。

2) 利用高级编程语言 Python 实现神经网络基本单元的前向传播（正向传播）和反向传播计算，加深对神经网络中基本单元的理解，包括全连接层、激活函数、损失函数等基本单元。

3) 利用高级编程语言 Python 实现神经网络训练所使用的梯度下降算法，加深对神经网络训练过程的理解。

实验工作量：约 20 行代码，约需 2 个小时。

2.1.2 背景知识

2.1.2.1 神经网络的组成

一个完整的神经网络通常由多个基本的网络层堆叠而成。本实验中的三层神经网络由三个全连接层构成，在每两个全连接层之间插入 ReLU 激活函数引入非线性变换，最后使用 Softmax 层计算交叉熵损失函数，如图2.1所示。因此本实验中使用的基本单元包括全连接层、ReLU 激活函数、Softmax 损失函数，将在本节分别介绍。更多关于神经网络中基本单元的介绍详见《智能计算系统》教材^[1]第 2.3 节。

1) 全连接层

全连接层以一维向量作为输入，输入与权重相乘后再与偏置相加得到输出向量。假设全连接层的输入为一维向量 $\mathbf{x}$ ，维度为 m ；输出为一维向量 $\mathbf{y}$ ，维度为 n ；权重是二维矩阵 $\mathbf{W}$ ，维度为 $m \times n$ ，偏置是一维向量 $\mathbf{b}^{\textcircled{1}}$ ，维度为 n 。前向传播时，全连接层的输出的计算公

^①偏置可以是一维向量，计算每个输出使用不同的偏置值；偏置也可以是一个标量，计算同一层的输出使用同一个偏置值。

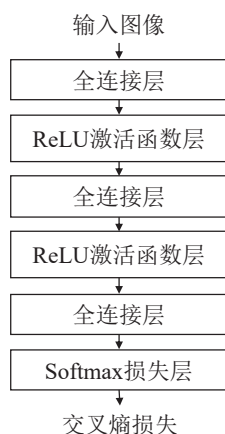


图 2.1 用于手写数字分类的三层全连接神经网络

式为

$$\mathbf{y} = \mathbf{W}^T \mathbf{x} + \mathbf{b} \quad (2.1)$$

在计算全连接层的反向传播时，给定神经网络损失函数 L 对当前全连接层的输出 $\mathbf{y}$ 的偏导 $\nabla_{\mathbf{y}} L = \frac{\partial L}{\partial \mathbf{y}}$ ，其维度与全连接层的输出 $\mathbf{y}$ 相同，均为 n 。根据链式法则，全连接层的权重和偏置的梯度 ($\nabla_{\mathbf{W}} L = \frac{\partial L}{\partial \mathbf{W}}$ 、 $\nabla_{\mathbf{b}} L = \frac{\partial L}{\partial \mathbf{b}}$) 以及损失函数对输入的偏导 ($\nabla_{\mathbf{x}} L = \frac{\partial L}{\partial \mathbf{x}}$) 的计算公式分别为：

$$\begin{aligned} \nabla_{\mathbf{W}} L &= \mathbf{x} \nabla_{\mathbf{y}} L^T \\ \nabla_{\mathbf{b}} L &= \nabla_{\mathbf{y}} L \\ \nabla_{\mathbf{x}} L &= \mathbf{W}^T \nabla_{\mathbf{y}} L \end{aligned} \quad (2.2)$$

实际应用中通常使用批量 (batch) 随机梯度下降算法进行反向传播计算，即选择若干个样本同时计算。假设选择的样本量为 p ，此时输入变为二维矩阵 $\mathbf{X}$ ，维度为 $p \times m$ ，每行代表一个样本。输出也变为二维矩阵 $\mathbf{Y}$ ，维度为 $p \times n$ 。此时全连接层的前向传播计算公式由公式(2.1)变为

$$\mathbf{Y} = \mathbf{XW} + \mathbf{b} \quad (2.3)$$

其中的 $+$ 代表广播运算，表示偏置 $\mathbf{b}$ 中的元素会被加到 $\mathbf{XW}$ 的乘积矩阵对应的一行元素中。权重和偏置的梯度 ($\nabla_{\mathbf{W}} L$ 、 $\nabla_{\mathbf{b}} L$) 以及损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 的计算公式由公式(2.2)变为

$$\begin{aligned} \nabla_{\mathbf{W}} L &= \mathbf{X}^T \nabla_{\mathbf{Y}} L \\ \nabla_{\mathbf{b}} L &= \mathbf{1} \nabla_{\mathbf{Y}} L \\ \nabla_{\mathbf{x}} L &= \nabla_{\mathbf{Y}} L \mathbf{W}^T \end{aligned} \quad (2.4)$$

其中计算偏置的梯度 $\nabla_{\mathbf{b}} L$ 时，为确保维度正确，用 $\nabla_{\mathbf{Y}} L$ 与维度为 $1 \times p$ 的全 1 向量 $\mathbf{1}$ 相乘。

2) ReLU 激活函数层

ReLU 激活函数是按元素运算操作，其输出向量 $\mathbf{y}$ 的维度与输入向量 $\mathbf{x}$ 的维度相同^①。在前向传播中，如果输入 $\mathbf{x}$ 中的元素值小于 0，则输出为 0，否则输出等于输入。因此 ReLU

^①输入和输出也可以是矩阵，处理方式类似。

的计算公式为

$$\mathbf{y}(i) = \max(0, \mathbf{x}(i)) \quad (2.5)$$

其中 $\mathbf{x}(i)$ 和 $\mathbf{y}(i)$ 分别代表 $\mathbf{x}$ 和 $\mathbf{y}$ 在位置 i 的值。

由于 ReLU 激活函数不包含参数，在反向传播计算过程中仅需根据损失函数对输出的偏导 $\nabla_{\mathbf{y}} L$ 计算损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 。损失函数对本层的第 i 个输入的偏导 $\nabla_{\mathbf{x}(i)} L$ 的计算公式为

$$\nabla_{\mathbf{x}(i)} L = \begin{cases} \nabla_{\mathbf{y}(i)} L, & \mathbf{x}(i) \geq 0 \\ 0, & \mathbf{x}(i) < 0 \end{cases} \quad (2.6)$$

3) Softmax 损失层

Softmax 损失层是目前多分类问题中最常用的损失函数层。假设 Softmax 损失层的输入为向量 $\mathbf{x}$ ，维度为 k 。其中 k 对应分类的类别数，如对手写数字 0 至 9 进行分类时，类别数 $k = 10$ 。

在前向传播的计算过程中，对 $\mathbf{x}$ 计算 e 指数并进行归一化，即得到 Softmax 分类概率。假设 $\mathbf{x}$ 在位置 i 的值为 $\mathbf{x}(i)$ ， $\hat{\mathbf{y}}(i)$ 为 Softmax 分类概率 $\hat{\mathbf{y}}$ 在位置 i 的值， $i \in [1, k]$ 且为整数，则 $\hat{\mathbf{y}}(i)$ 的计算公式为

$$\hat{\mathbf{y}}(i) = \frac{e^{\mathbf{x}(i)}}{\sum_j e^{\mathbf{x}(j)}} \quad (2.7)$$

在神经网络推断时，完成 Softmax 损失层的前向传播之后，取 Softmax 分类概率 $\hat{\mathbf{y}}$ 中最大概率对应的类别作为预测的分类类别。

在神经网络训练时，在上述前向传播基础上，Softmax 损失层还需要根据给定的标记 (label，也称为真实值或实际值) 计算总的损失函数值。在分类任务中，标记通常表示为一个维度为 k 的 one-hot 向量 $\mathbf{y}$ ，该向量中对应真实类别的分量值为 1，其他值为 0。Softmax 损失层使用交叉熵损失函数 L ，其计算公式为

$$L = - \sum_i \mathbf{y}(i) \ln \hat{\mathbf{y}}(i) \quad (2.8)$$

在反向传播的计算过程中，对于 Softmax 损失层，损失函数对输入的偏导 $\nabla_{\mathbf{x}} L$ 的计算公式为

$$\nabla_{\mathbf{x}} L = \frac{\partial L}{\partial \mathbf{x}} = \hat{\mathbf{y}} - \mathbf{y} \quad (2.9)$$

由于工程实现中使用批量随机梯度下降算法，假设选择的样本量为 p ，Softmax 损失层的输入变为二维矩阵 $\mathbf{X}$ ，维度为 $p \times k$ ， $\mathbf{X}$ 的每个行向量代表一个样本，则对每个输入计算 e 指数并进行行归一化得到

$$\hat{\mathbf{Y}}(i, j) = \frac{e^{\mathbf{X}(i, j)}}{\sum_j e^{\mathbf{X}(i, j)}} \quad (2.10)$$

其中 $\mathbf{X}(i, j)$ 代表 $\mathbf{X}$ 中对应第 i 个样本 j 位置的值。当 $\mathbf{X}(i, j)$ 数值较大时，求 e 指数可能会出现数值上溢的问题。因此在实际工程实现时，为确保数值稳定性，会在求 e 指数前进行减最大值处理，此时 $\hat{\mathbf{Y}}(i, j)$ 的计算公式变为

$$\hat{\mathbf{Y}}(i, j) = \frac{e^{\mathbf{X}(i, j) - \max_n \mathbf{X}(i, n)}}{\sum_j e^{\mathbf{X}(i, j) - \max_n \mathbf{X}(i, n)}} \quad (2.11)$$

在神经网络推断时，取 Softmax 分类概率 $\hat{Y}(i, j)$ 的每个样本（即每个行向量）中最大概率对应的类别作为预测的分类类别。

做神经网络训练时，标记通常表示为一组 one-hot 向量 Y ，维度为 $p \times k$ ，其中每行是一个 one-hot 向量，对应一个样本的标记。则计算损失值的公式(2.8)变为

$$L = -\frac{1}{p} \sum_{i,j} Y(i, j) \ln \hat{Y}(i, j) \quad (2.12)$$

其中损失值是所有样本的平均损失，因此对样本数量 p 取平均。

在反向传播时，当选择的样本量为 p 时，损失函数对输入的偏导 $\nabla_x L$ 的计算公式(2.9)变为

$$\nabla_x L = \frac{1}{p} (\hat{Y} - Y) \quad (2.13)$$

2.1.2.2 神经网络训练

神经网络训练通过调整网络层的参数来使神经网络计算出来的结果与真实结果（标记）尽量接近。神经网络训练通常使用随机梯度下降算法，通过不断地迭代计算每层参数的梯度，利用梯度对每层参数进行更新。具体而言，给定当前迭代的训练样本（包含输入数据及标记信息），首先进行神经网络的前向传播处理，即输入数据和权重相乘再经过激活函数计算出隐层，隐层神经元与下一层的权重相乘再经过激活函数得到下一个隐层，通过逐层迭代计算出神经网络的输出结果。随后利用输出结果和标记信息计算出损失函数值。然后进行神经网络的反向传播处理，从损失函数开始逆序逐层计算损失函数对权重和偏置的偏导（即梯度），最后利用梯度对相应的参数进行更新。更新参数 W 的计算公式为

$$W \leftarrow W - \eta \nabla_w L \quad (2.14)$$

其中， $\nabla_w L$ 为参数的梯度， η 是学习率。

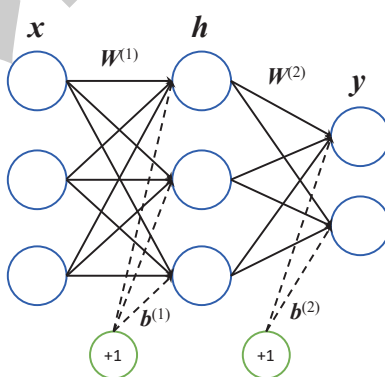


图 2.2 两层神经网络示例

下面以图2.2中的两层神经网络为例，介绍神经网络训练的具体过程。图2.2中的网络由两个全连接层及 Softmax 损失层组成^①，其中第一个全连接层的权重为 $W^{(1)}$ ，偏置为 $b^{(1)}$ ，

^①本实验中实现的三层神经网络与这个例子非常类似，即在这个例子基础上再添加一层全连接层和一层 ReLU 层即可，网络训练的过程也与这个例子完全一致

第二个全连接层的权重为 $\mathbf{W}^{(2)}$ ，偏置为 $\mathbf{b}^{(2)}$ 。假设某次迭代的神经网络输入为 $\mathbf{x}$ ，对应的标记为 $\mathbf{y}$ 。该神经网络前向传播的逐层计算公式依次是

$$\begin{aligned}\mathbf{h} &= \mathbf{W}^{(1)T} \mathbf{x} + \mathbf{b}^{(1)} \\ \mathbf{z} &= \mathbf{W}^{(2)T} \mathbf{h} + \mathbf{b}^{(2)}\end{aligned}\quad (2.15)$$

其中 $\mathbf{h}, \mathbf{z}$ 分别是第一、第二层全连接层的输出。Softmax 损失层的损失值 L 为：

$$\begin{aligned}\hat{y}(i) &= \frac{e^{z(i)}}{\sum_i e^{z(i)}} \\ L &= -\sum_i y(i) \ln \hat{y}(i)\end{aligned}\quad (2.16)$$

反向传播的逐层计算公式为

$$\begin{aligned}\nabla_{\mathbf{z}} L &= \hat{\mathbf{y}} - \mathbf{y} \\ \nabla_{\mathbf{W}^{(2)}} L &= \mathbf{h} \nabla_{\mathbf{z}} L^T \\ \nabla_{\mathbf{b}^{(2)}} L &= \nabla_{\mathbf{z}} L \\ \nabla_{\mathbf{h}} L &= \mathbf{W}^{(2)T} \nabla_{\mathbf{z}} L \\ \nabla_{\mathbf{W}^{(1)}} L &= \mathbf{x} \nabla_{\mathbf{h}} L^T \\ \nabla_{\mathbf{b}^{(1)}} L &= \nabla_{\mathbf{h}} L \\ \nabla_{\mathbf{x}} L &= \mathbf{W}^{(1)T} \nabla_{\mathbf{h}} L\end{aligned}\quad (2.17)$$

其中 $\nabla_{\mathbf{z}} L$ 、 $\nabla_{\mathbf{h}} L$ 、 $\nabla_{\mathbf{x}} L$ 分别是损失函数对 Softmax 层、第二层、第一层的偏导， $\nabla_{\mathbf{W}^{(2)}} L$ 、 $\nabla_{\mathbf{b}^{(2)}} L$ 、 $\nabla_{\mathbf{W}^{(1)}} L$ 、 $\nabla_{\mathbf{b}^{(1)}} L$ 分别是第二层和第一层的权重和偏置梯度， η 为学习率。更新两个全连接层的权重和偏置的计算为

$$\begin{aligned}\mathbf{W}^{(1)} &\leftarrow \mathbf{W}^{(1)} - \eta \nabla_{\mathbf{W}^{(1)}} L \\ \mathbf{b}^{(1)} &\leftarrow \mathbf{b}^{(1)} - \eta \nabla_{\mathbf{b}^{(1)}} L \\ \mathbf{W}^{(2)} &\leftarrow \mathbf{W}^{(2)} - \eta \nabla_{\mathbf{W}^{(2)}} L \\ \mathbf{b}^{(2)} &\leftarrow \mathbf{b}^{(2)} - \eta \nabla_{\mathbf{b}^{(2)}} L\end{aligned}\quad (2.18)$$

神经网络训练相关的详细介绍可以参见《智能计算系统》教材第 2.2 节。

2.1.2.3 精度评估

在图像分类任务中，通常使用测试集的平均分类正确率来判断分类结果的精度。假设共有 N 个图像样本（MNIST 手写数据集中共包含 10000 张测试图像，此时 $N = 10000$ ），神经网络推断出的第 i 张图像的分类结果为 $\hat{\mathbf{C}}(i)$ ，第 i 张图像的标记（即实际类别）为 $\mathbf{C}(i)$ ，则平均分类正确率 R 的计算公式为

$$R = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{\mathbf{C}}(i) = \mathbf{C}(i)) \quad (2.19)$$

其中 $\mathbf{1}(\hat{\mathbf{C}}(i) = \mathbf{C}(i))$ 表示当第 i 张图像的推断出的类别与标记相等时值为 1，否则值为 0。

2.1.3 实验环境

硬件环境：CPU。

软件环境:Python 编译环境及相关的扩展库, 包括 Python 3.6.12、Pillow 7.2.0、Scipy1.2.0、NumPy 1.19.5(本实验不需使用 Pytorch 等深度学习框架)。

数据集: MNIST 手写数字库^[2]。该数据集包含一个训练集和一个测试集, 其中训练集有 60000 个样本, 测试集有 10000 个样本。每个样本都由灰度图像(即单通道图像)及其标记组成, 每个样本图像的大小为 28×28 。MNIST 数据集包含 4 个文件, 分别是训练集图像、训练集标记、测试集图像、测试集标记。下载地址为<http://yann.lecun.com/exdb/mnist/>。

2.1.4 实验内容

设计一个三层神经网络实现手写数字图像分类。该网络包含两个隐层和一个输出层, 其中输入神经元个数由输入数据维度决定, 输出层的神经元个数由数据集包含的类别决定, 两个隐层的神经元个数可以作为超参数自行设置。对于手写数字图像的分类问题, 神经网络的输入数据为手写数字图像。原始数字图像一般用二维矩阵(灰度图像)或三维矩阵(彩色图像)来表示, 其在输入神经网络前会调整为一维向量形式。神经网络的输出神经元个数为待分类的类别数, 一般是提前预设的, 如手写数字包含 0 至 9 共 10 个类别。

为了便于迭代开发, 工程实现时采用模块化的方式来实现整个神经网络的处理。目前绝大多数神经网络的工程实现通常划分为 5 大模块:

- 1) 数据加载模块: 从文件中读取数据, 并进行预处理, 其中预处理包括归一化、维度变换等处理。如果需要人为对数据进行随机数据扩增, 则数据扩增处理也在数据加载模块中实现。
- 2) 基本单元模块: 实现神经网络中不同类型的网络层的定义、前向传播计算、反向传播计算等功能。
- 3) 网络结构模块: 利用基本单元模块建立一个完整的神经网络。
- 4) 网络训练模块: 该模块实现用训练集进行神经网络训练的功能。在已建立的神经网络结构基础上, 实现神经网络的前向传播、神经网络的反向传播、神经网络参数更新、神经网络参数保存等基本操作, 以及训练函数主体。
- 5) 网络推断模块: 该模块实现使用训练得到的网络模型, 对测试样本进行预测的过程。具体实现的操作包括训练得到的网络模型参数的加载、神经网络的前向传播等。

本实验采用上述较为细致的模块划分方式。需要说明的是, 目前有些开源的神经网络工程可能采用较粗的模块划分方式, 例如将基本单元模块与网络结构模块合并, 或将网络训练模块与网络推断模块合并。在某些特殊的应用场景下, 神经网络工程可能不需要包含所有模块, 例如仅实现推断过程的工程通常不包含训练模块(如实验3.1), 非实时风格迁移工程不包含推断模块(如实验3.3)。

2.1.5 实验步骤

本节介绍如何实现本实验涉及各个模块, 以及如何搭建和调用各个模块来实现手写数字图像分类。

2.1.5.1 数据加载模块

本实验采用的数据集是 MNIST 手写数字库^[2]。该数据集中的图像数据和标记数据采用表2.1中的 IDX 文件格式存放。图像的像素值按行优先顺序存放，取值范围为 [0,255]，其中 0 表示黑色，255 表示白色。

表 2.1 MNIST 数据集 IDX 文件格式^[2]

图像文件格式			
字节偏移	数据类型	值	描述
0000	int32 (32 位有符号整型)	0x00000803(2051)	magic number (魔数): 表示像素的数据类型以及像素数据的维度信息, MSB (大尾端)
0004	int32	60000 (训练集) 10000 (测试集)	图像数量
0008	int32	28	图像行数, 即图像高度
0012	int32	28	图像列数, 即图像宽度
0016	uint8 (8 位无符号整型)	??	像素值
0017	uint8	??	像素值
.....			
xxxx	uint8	??	像素值
标记文件格式			
字节偏移	数据类型	值	描述
0000	int32	0x00000801(2049)	magic number
0004	int32	60000 (训练集) 10000 (测试集)	标记数量
0008	uint8	??	标记
0009	uint8	??	标记
.....			
xxxx	uint8	??	标记

首先编写读取 MNIST 数据集文件并预处理的子函数，如代码示例2.1所示。

代码示例 2.1 MNIST 数据集文件的读取和预处理

```

1 # file: mnist_mlp_cpu.py
2 def load_mnist(self, file_dir, is_images = 'True'):
3     bin_file = open(file_dir, 'rb')
4     bin_data = bin_file.read()
5     bin_file.close()
6     if is_images: # 读取图像数据
7         fmt_header = '>iiii'
8         magic, num_images, num_rows, num_cols = struct.unpack_from(fmt_header, bin_data, 0)
9     else: # 读取标记数据
10        fmt_header = '>ii'
11        magic, num_images = struct.unpack_from(fmt_header, bin_data, 0)
12        num_rows, num_cols = 1, 1
13        data_size = num_images * num_rows * num_cols
14        mat_data = struct.unpack_from('>' + str(data_size) + 'B', bin_data, struct.calcsize(fmt_header))
15        mat_data = np.reshape(mat_data, [num_images, num_rows * num_cols])
16    return mat_data

```

然后调用该子函数对 MNIST 数据集中的 4 个文件分别进行读取和预处理，并将处理过的训练和测试数据存储在 NumPy 矩阵中（训练模型时可以快速读取该矩阵中的数据），如代码示例2.2所示。

代码示例 2.2 MNIST 子数据集的读取和预处理

```
1 # file: mnist_mlp_cpu.py
2 def load_data(self):
3     # TODO: 调用函数 load_mnist 读取和预处理 MNIST 中训练数据和测试数据的图像和标记
4     train_images = self.load_mnist(os.path.join(MNIST_DIR, TRAIN_DATA), True)
5     train_labels = _____
6     test_images = _____
7     test_labels = _____
8     self.train_data = np.append(train_images, train_labels, axis=1)
9     self.test_data = np.append(test_images, test_labels, axis=1)
```

2.1.5.2 基本单元模块

本实验采用图2.1所示的三层神经网络，主体是三个全连接层。在前两个全连接层之后均使用 ReLU 激活函数层引入非线性变换，在神经网络的最后添加 Softmax 层计算交叉熵损失。因此，本实验中需要实现的基本单元模块包括全连接层、ReLU 激活函数层和 Softmax 损失层。

在神经网络实现中，通常同类型的层用一个类来定义，多个同类型的层用类的实例来实现，层内的计算用类的成员函数来定义。类的成员函数通常包括层的初始化、参数的初始化、前向传播计算、反向传播计算、参数的更新、参数的加载和保存等。其中层的初始化函数一般会根据实例化层时的输入确定该层的超参数，例如该层的输入神经元数量和输出神经元数量等；参数的初始化函数会对该层的参数（如全连接层中的权重和偏置）分配存储空间，并填充初始值；前向传播函数利用前一层的输出作为本层的输入，计算本层的输出结果；反向传播函数根据链式法则逆序逐层计算损失函数对权重和偏置的梯度；参数的更新函数利用反向传播函数计算的梯度对本层的参数进行更新；参数的加载函数从给定的文件中加载参数的值，参数的保存函数将当前层参数的值保存到指定的文件中。有些层（如激活函数层）可能没有参数，就不需要定义参数的初始化、更新、加载和保存函数。有些层（如激活函数层和损失函数层）的输出维度由输入维度决定，不需要人工设定，因此不需要层的初始化函数。

以下是全连接层、ReLU 激活函数层和 Softmax 损失层的具体实现步骤。

1) **全连接层**：实现如代码示例2.3所示，定义了以下成员函数：

- 层的初始化：需要确定该全连接层的输入神经元个数（即输入二维矩阵中每个行向量的维度）和输出神经元个数（即输出二维矩阵中每个行向量的维度）。
- 参数初始化：全连接层的参数包括权重和偏置。根据输入神经元个数 m 和输出神经元个数 n 可以确定权重 W 的维度为 $m \times n$ ，偏置 b 的维度为 n 。在对权重和偏置进行初始化时，通常利用高斯随机数来初始化权重的值，而将偏置的所有值初始化为 0。
- 前向传播计算：全连接层的前向传播计算公式为公式(2.3)，可以通过输入矩阵与权重矩阵相乘再与偏置相加实现。
- 反向传播计算：全连接层的反向传播计算公式为公式(2.4)。给定损失函数对本层输出的偏导 $\nabla_Y L$ ，利用矩阵相乘计算权重和偏置的梯度 $\nabla_W L$ 、 $\nabla_b L$ 以及损失函数对本层输入的偏导 $\nabla_X L$ 。
- 参数更新：给定学习率 η ，利用反向传播计算得到的权重梯度 $\nabla_W L$ 和偏置梯度 $\nabla_b L$

对本层的权重 $\mathbf{W}$ 和偏置 $\mathbf{b}$ 进行更新：

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L \quad (2.20)$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \nabla_{\mathbf{b}} L \quad (2.21)$$

- 参数加载：从该函数的输入中读取本层的权重 $\mathbf{W}$ 和偏置 $\mathbf{b}$ 。
- 参数保存：返回本层当前的权重 $\mathbf{W}$ 和偏置 $\mathbf{b}$ 。

代码示例 2.3 全连接层的实现

```

1 # file: layers_1.py
2 class FullyConnectedLayer(object):
3     def __init__(self, num_input, num_output): # 全连接层初始化
4         self.num_input = num_input
5         self.num_output = num_output
6     def init_param(self, std=0.01): # 参数初始化
7         self.weight = np.random.normal(loc=0.0, scale=std, size=(self.num_input, self.num_output))
8         self.bias = np.zeros([1, self.num_output])
9     def forward(self, input): # 前向传播的计算
10        self.input = input
11        # TODO: 全连接层的前向传播, 计算输出结果
12        self.output = _____
13        return self.output
14    def backward(self, top_diff): # 反向传播的计算
15        # TODO: 全连接层的反向传播, 计算参数梯度和本层损失
16        self.d_weight = _____
17        self.d_bias = _____
18        bottom_diff = _____
19        return bottom_diff
20    def update_param(self, lr): # 参数更新
21        # TODO: 利用梯度对全连接层参数进行更新
22        self.weight = _____
23        self.bias = _____
24    def load_param(self, weight, bias): # 参数加载
25        self.weight = weight
26        self.bias = bias
27    def save_param(self): # 参数保存
28        return self.weight, self.bias

```

2) **ReLU 激活函数层**：ReLU 层不包含参数，因此实现中没有参数初始化、参数更新、参数的加载和保存相关的函数。ReLU 层的实现如代码示例2.4所示，定义了以下成员函数：

- 前向传播计算：根据公式(2.5)可以计算 ReLU 层前向传播的结果。在工程实现中，可以对整个输入矩阵使用 maximum 函数，maximum 函数会进行广播，计算输入矩阵的每个元素与 0 的最大值。
- 反向传播计算：根据公式(2.6)可以计算损失函数对输入的偏导。在工程实现中，可以获取 $x(i) < 0$ 的位置索引，将 $\nabla_{\mathbf{x}} L$ 中对应位置的值置为 0。

代码示例 2.4 ReLU 激活函数层的实现，其中 TODO 表示需要读者来实现的内容

```

1 # file: layers_1.py
2 class ReLULayer(object):
3     def forward(self, input): # 前向传播的计算
4         self.input = input
5         # TODO: ReLU 层的前向传播, 计算输出结果
6         output = _____
7         return output

```

```

8 def backward(self, top_diff): # 反向传播的计算
9     # TODO: ReLU 层的反向传播, 计算本层损失
10    bottom_diff = -----
11    return bottom_diff

```

3) **Softmax 损失层**: 同样不包含参数, 因此实现中没有参数初始化、更新、加载和保存相关的函数。但该层需要计算总的损失函数值, 作为训练时的中间输出结果, 帮助判断模型的训练进程。Softmax 损失层的实现如代码示例2.5所示, 定义了以下成员函数:

- 前向传播计算: 可以使用公式(2.11)计算。该公式为确保数值稳定, 会在求 e 指数前做减最大值处理。
- 损失函数计算: 可以使用公式(2.12)计算, 采用批量随机梯度下降法训练时损失值是 batch 内所有样本的损失值的均值。需要注意的是, MNIST 手写数字库的标记数据读入的是 0 至 9 的类别编号, 在计算损失时需要先将类别编号转换为 one-hot 向量。
- 反向传播计算: 可以使用公式(2.13)计算, 计算时同样需要对样本数量取平均。

代码示例 2.5 Softmax 损失层的实现

```

1 # file: layers_1.py
2 class SoftmaxLossLayer(object):
3     def forward(self, input): # 前向传播的计算
4         # TODO: Softmax 损失层的前向传播, 计算输出结果
5         input_max = np.max(input, axis=1, keepdims=True)
6         input_exp = np.exp(input - input_max)
7         self.prob = -----
8         return self.prob
9     def get_loss(self, label): # 计算损失
10        self.batch_size = self.prob.shape[0]
11        self.label_onehot = np.zeros_like(self.prob)
12        self.label_onehot[np.arange(self.batch_size), label] = 1.0
13        loss = -np.sum(np.log(self.prob) * self.label_onehot) // self.batch_size
14        return loss
15    def backward(self): # 反向传播的计算
16        # TODO: Softmax 损失层的反向传播, 计算本层损失
17        bottom_diff = -----
18        return bottom_diff

```

2.1.5.3 网络结构模块

网络结构模块利用已经实现的神经网络的基本单元来建立一个完整的神经网络。在工程实现中通常用一个类来定义一个神经网络, 用类的成员函数来定义神经网络的初始化、建立神经网络结构、神经网络参数初始化等基本操作。本实验中三层神经网络的网络结构模块的实现如代码示例2.6所示, 定义了以下成员函数:

- 神经网络初始化: 确定神经网络相关的超参数, 例如网络中每个隐层的神经元个数。
- 建立网络结构: 定义整个神经网络的拓扑结构, 实例化基本单元模块中定义的层并将这些层进行堆叠。例如本实验使用的三层神经网络包含三个全连接层, 并且在前两个全连接层后都跟随有 ReLU 层, 神经网络的最后使用了 Softmax 损失层。
- 神经网络参数初始化: 对于神经网络中包含参数的层, 依次调用这些层的参数初始化函数, 从而完成整个神经网络的参数初始化。本实验使用的三层神经网络中, 只有三个全连接层包含参数, 依次调用其参数初始化函数即可。

代码示例 2.6 三层神经网络的网络结构模块实现

```

1 # file: mnist_mlp_cpu.py
2 class MNIST_MLP(object):
3     def __init__(self, batch_size=100, input_size=784, hidden1=32, hidden2=16, out_classes=10, lr
4         =0.01, max_epoch=2, print_iter=100):
5         # 神经网络初始化
6         self.batch_size = batch_size
7         self.input_size = input_size
8         self.hidden1 = hidden1
9         self.hidden2 = hidden2
10        self.out_classes = out_classes
11        self.lr = lr
12        self.max_epoch = max_epoch
13        self.print_iter = print_iter
14    def build_model(self): # 建立网络结构
15        # TODO: 建立三层神经网络结构
16        self.fc1 = FullyConnectedLayer(self.input_size, self.hidden1)
17        self.relu1 = ReLULayer()
18
19        self.fc3 = FullyConnectedLayer(self.hidden2, self.out_classes)
20        self.softmax = SoftmaxLossLayer()
21        self.update_layer_list = [self.fc1, self.fc2, self.fc3]
22    def init_model(self): # 神经网络参数初始化
23        for layer in self.update_layer_list:
24            layer.init_param()

```

2.1.5.4 网络训练模块

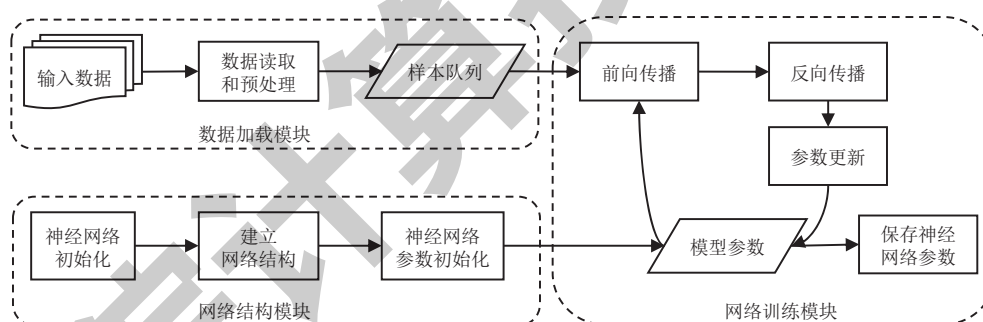


图 2.3 神经网络训练流程

神经网络训练流程如图2.3所示。在完成数据加载模块和网络结构模块实现之后，需要实现网络训练模块。本实验中三层神经网络的网络训练模块的实现如代码示例2.7所示。神经网络的训练模块通常拆解为若干步骤，包括神经网络的前向传播、神经网络的反向传播、神经网络参数更新、神经网络参数保存等基本操作。这些网络训练模块的基本操作以及训练主体用神经网络类的成员函数来定义：

- 神经网络的前向传播：根据神经网络的拓扑结构，顺序调用每层的前向传播函数。以输入数据作为第一层的输入，之后每层的输出作为其下一层的输入，顺序计算每一层的输出，最后得到损失函数层的输出。
- 神经网络的反向传播：根据神经网络的拓扑结构，逆序调用每层的反向传播函数。采用链式法则逆序逐层计算损失函数对每层参数的偏导，最后得到神经网络所有层的参数梯度。

• 神经网络参数更新：对神经网络中包含参数的层，依次调用各层的参数更新函数，来对整个神经网络的参数进行更新。本实验使用的三层神经网络中只有三个全连接层包含参数，因此依次更新三个全连接层的参数即可。

• 神经网络参数保存：对神经网络中包含参数的层，依次收集这些层的参数并存储到文件中。

• 神经网络训练主体：在该函数中，(1) 确定训练的一些超参数，如使用批量梯度下降算法时的批量大小、学习率大小、迭代次数（或训练周期次数）、可视化训练过程时每迭代多少次屏幕输出一次当前的损失值等等。(2) 开始迭代训练过程。每次迭代训练开始前，可以根据需要对数据进行随机打乱，一般是一个训练周期（即当整个数据集的数据都参与一次训练过程）后对数据集进行随机打乱。每次迭代训练过程中，先选取当前迭代所使用的数据和对应的标记，再进行整个网络的前向传播，随后计算当前迭代的损失值，然后进行整个网络的反向传播来获得整个网络的参数梯度，最后对整个网络的参数进行更新。完成一次迭代后可以根据需要在屏幕上输出当前的损失值，以供实际应用中修改模型参考。完成神经网络的训练过程后，通常会将训练得到的神经网络模型参数保存到文件中。

代码示例 2.7 三层神经网络的训练模块实现

```
1 # file: mnist_mlp_cpu.py
2 def forward(self, input): # 神经网络的前向传播
3     # TODO: 神经网络的前向传播
4     h1 = self.fc1.forward(input)
5     h1 = self.relu1.forward(h1)
6     -----
7     prob = self.softmax.forward(h3)
8     return prob
9
10 def backward(self): # 神经网络的反向传播
11     # TODO: 神经网络的反向传播
12     dloss = self.softmax.backward()
13     -----
14     dh1 = self.relu1.backward(dh2)
15     dh1 = self.fc1.backward(dh1)
16
17 def update(self, lr): # 神经网络参数更新
18     for layer in self.update_layer_list:
19         layer.update_param(lr)
20
21 def save_model(self, param_dir): # 保存神经网络参数
22     params = {}
23     params['w1'], params['b1'] = self.fc1.save_param()
24     params['w2'], params['b2'] = self.fc2.save_param()
25     params['w3'], params['b3'] = self.fc3.save_param()
26     np.save(param_dir, params)
27
28 def train(self): # 训练函数主体
29     max_batch = self.train_data.shape[0] // self.batch_size
30     for idx_epoch in range(self.max_epoch):
31         mlp.shuffle_data()
32         for idx_batch in range(max_batch):
33             batch_images = self.train_data[idx_batch*self.batch_size:(idx_batch+1)*self.batch_size,
34                                     :-1]
35             batch_labels = self.train_data[idx_batch*self.batch_size:(idx_batch+1)*self.batch_size,
36                                     -1]
37
38             prob = self.forward(batch_images)
39             loss = self.softmax.get_loss(batch_labels)
```

```

37         self.backward()
38         self.update(self.lr)
39         if idx_batch % self.print_iter == 0:
40             print('Epoch %d, iter %d, loss: %.6f' % (idx_epoch, idx_batch, loss))

```

2.1.5.5 网络推断模块

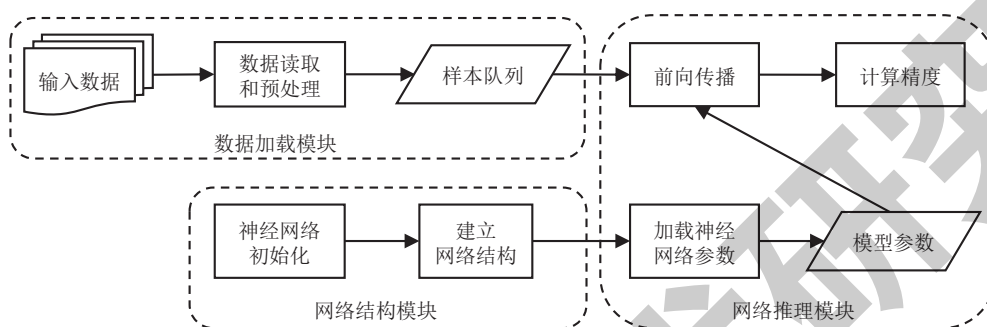


图 2.4 神经网络推断流程

整个神经网络推断流程如图2.4所示。完成神经网络的训练之后，可以用训练得到的模型对测试数据进行预测，以评估模型的精度。本实验中三层神经网络的推断模块的实现如代码示例2.8所示。工程实现中同样常将一个神经网络的推断模块拆解为若干步骤，包括神经网络参数加载、神经网络的前向传播等基本操作。这些网络推断模块的基本操作以及推断主体用神经网络类的成员函数来定义：

- 神经网络的前向传播：网络推断模块中的神经网络前向传播操作与网络训练模块中的前向传播操作完全一致，因此可以直接调用网络训练模块中的神经网络前向传播函数。
- 神经网络参数加载：读取神经网络训练模块保存的模型参数文件，并加载有参数的网络层的参数值。
- 神经网络推断函数主体：在进行神经网络推断前，需要从模型参数文件中加载神经网络的参数。在神经网络推断过程中，循环每次读取一定批量的测试数据，随后进行整个神经网络的前向传播计算得到神经网络的输出结果，对于每个样本取输出结果中最大概率对应的类别作为预测的分类类别，再与测试数据集的标记进行比对，利用相关的评价函数计算模型的精度，如手写数字分类问题使用分类平均正确率作为模型的评价函数。

代码示例 2.8 三层神经网络的推断模块实现

```

1 # file: mnist_mlp_cpu.py
2 def load_model(self, param_dir): # 加载神经网络参数
3     params = np.load(param_dir, allow_pickle=True, encoding="latin1").item()
4     self.fc1.load_param(params['w1'], params['b1'])
5     self.fc2.load_param(params['w2'], params['b2'])
6     self.fc3.load_param(params['w3'], params['b3'])
7
8 def evaluate(self): # 推断函数主体
9     pred_results = np.zeros([self.test_data.shape[0]])
10    for idx in range(self.test_data.shape[0]//self.batch_size):
11        batch_images = self.test_data[idx*self.batch_size:(idx+1)*self.batch_size, :-1]
12        prob = self.forward(batch_images)
13        pred_labels = np.argmax(prob, axis=1)

```



```
14     pred_results[idx*self.batch_size:(idx+1)*self.batch_size] = pred_labels
15     accuracy = np.mean(pred_results == self.test_data[:, -1])
16     print('Accuracy in test set: %f' % accuracy)
```

2.1.5.6 完整实验流程

完成神经网络的各个模块之后，就可以调用这些模块实现用三层神经网络进行手写数字图像分类的完整流程，如代码示例2.9所示。首先实例化三层神经网络对应的类，指定神经网络的超参数，如每层的神经元个数。其次进行数据的加载和预处理。再调用网络结构模块建立神经网络，随后进行网络初始化，在该过程中网络结构模块会自动调用基本单元模块实例化神经网络中的每个层。然后调用网络训练模块训练整个网络，之后将训练得到的模型参数保存到文件中。最后从文件中读取训练得到的模型参数，调用网络推断模块测试网络的精度。

代码示例 2.9 三层神经网络的完整流程实现

```
1 # file: mnist_mlp_cpu.py
2 def build_mnist_mlp(param_dir='weight.npy'):
3     h1, h2, e = 32, 16, 10
4     mlp = MNIST_MLP(hidden1=h1, hidden2=h2, max_epoch=e)
5     mlp.load_data()
6     mlp.build_model()
7     mlp.init_model()
8     mlp.train()
9     mlp.save_model('mlp-%d-%d-%d-depoch.npy' % (h1, h2, e))
10    mlp.load_model('mlp-%d-%d-%d-depoch.npy' % (h1, h2, e))
11    return mlp
12
13 if __name__ == '__main__':
14     mlp = build_mnist_mlp()
15     mlp.evaluate()
```

2.1.5.7 实验运行

根据第2.1.5.1节 ~ 第2.1.5.6节的描述补全 layer_1.py、mnist_mlu_cpu.py 代码，并通过 Python 运行.py 代码。具体可以参考以下步骤。

1) 环境申请

申请实验环境并登录云平台，云平台上/opt/code_chap_2_3/code_chap_2_3_student/ 目录下是本实验的示例代码。

```
# 登录云平台
ssh root@xxx.xxx.xxx.xxx -p xxxxx
# 进入 code_chap_2_3_student 目录
cd /opt/code_chap_2_3/code_chap_2_3_student
```

2) 代码实现

如下所示，补全 stu_upload 中的 layers_1.py, mnist_mlp_cpu.py 文件。

```
# 进入实验目录
cd exp_2_1_mnist_mlp
# 补全 layers_1.py, mnist_mlp_cpu.py
```

```
vim stu_upload/layers_1.py
vim stu_upload/mnist_mlp_cpu.py
```

3) 运行实验

```
# 运行完整实验
python main_exp_2_1.py
```

2.1.6 实验评估

本实验的评估标准设定如下：

- 60 分标准：给定全连接层、ReLU 层、Softmax 损失层的前向传播的输入矩阵、参数值、反向传播的输入，可以得到正确的前向传播的输出矩阵、反向传播的输出和参数梯度。
- 80 分标准：实现正确的三层神经网络，并进行训练和推断，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 92%。
- 90 分标准：实现正确的三层神经网络，并进行训练和推断，通过调整与训练相关的超参数，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 95%。
- 100 分标准：在三层神经网络基础上设计自己的神经网络结构，并进行训练和推断，使最后训练得到的模型在 MNIST 测试数据集上的平均分类正确率高于 98%。

2.1.7 实验思考

- 1) 在实现神经网络基本单元时，如何确保一个网络层的实现是正确的？
- 2) 在实现神经网络后，如何在不改变网络结构的条件下提高精度？
- 3) 如何通过修改网络结构提高精度？可以从哪些方面修改网络结构？

2.2 基于 DLP 平台实现手写数字分类

2.2.1 实验目的

熟悉深度学习处理器 DLP 平台的使用，能使用已封装好的 Python 接口的机器学习编程库 pycnnl 将第2.1节的神经网络推断部分移植到 DLP 平台，实现手写数字分类。具体包括：

- 1) 利用 pycnnl 库中的 Python 接口搭建手写数字分类的三层神经网络。
- 2) 熟悉在 DLP 上运行神经网络的流程，为在后续章节详细学习 DLP 高性能库以及智能编程语言打下基础。
- 3) 与第2.1节的实验进行比较，了解 DLP 相对于 CPU 的优势和劣势。

实验工作量：约需 2 个小时。

2.2.2 背景知识

2.2.2.1 Python 接口的深度学习编程库 pycnnl

深度学习编程库 pycnnl 通过调用 DLP 上的 CNL 库中的高性能算子实现了全连接层、卷积层、池化层、ReLU 激活层、Softmax 损失层等常用的网络层的基本功能，并提供了常

用网络层的 Python 接口。pycnnl 提供的编程接口可以用于在 DLP 上加速神经网络算法，具体接口说明如表2.2所示。pycnnl 用 Python 封装了一个 C++ 类 CnnlNet，该类的成员函数定义了神经网络中层的创建、网络前向传播、参数加载等操作。

表 2.2 pycnnl 接口说明

接口	功能描述	参数/返回值
setInputShape: pycnnl.CnnlNet().setInputShape(dim_1, dim_2, dim_3, dim_4)	设定网络第一层输入数据的形状	dim_1 (int): 维度 1 dim_2 (int): 维度 2 dim_3 (int): 维度 3 dim_4 (int): 维度 4
createConvLayer: pycnnl.CnnlNet().createConvLayer(layer_name, input_shape, output_channel, kernel_size, stride, dilation, pad)	创建卷积层	layer_name: 该层的名称 input_shape(list): 输入数据的形状, [N, input channel, input height, input width] output_channel (int): 输出 channel 的大小 kernel_size (int): 卷积核的大小 stride (int): 卷积步长 dilation (int): 膨胀系数 pad (int): 填充大小
createMlpLayer: pycnnl.CnnlNet().createMlpLayer(layer_name, input_shape, weight_shape, output_shape)	创建全连接层	layer_name: 该层的名称 input_shape(list): 输入数据的形状, [N, input height, input weight, input channel] weight_shape (list): 输入数据的形状, [N, weight height, input channel, output channel] output_shape (list): 输入数据的形状, [N, output weight, output width, output channel]
createReLuLayer: pycnnl.CnnlNet().createReLuLayer(layer_name)	创建 ReLU 激活函数层	layer_name: 该层的名称
createSoftmaxLayer: pycnnl.CnnlNet().createSoftmaxLayer(layer_name, input_shape, axis=1)	创建 Softmax 损失层	layer_name: 该层的名称 input_shape : 输入数据的形状: batch_size, 1, output_classes axis (int): 进行 Softmax 计算的维度
createPoolingLayer: pycnnl.CnnlNet().createPoolingLayer(layer_name, input_shape, kernel_size, stride)	创建最大池化层	layer_name: 该层的名称 input_shape : 输入数据的形状 kernel_size (int): pool 窗口的大小 stride (int): 窗口滑动步长
loadParams: pycnnl.CnnlNet().loadParams(layer_id, filter_data, bias_data)	为指定的层加载参数	layer_id (int): 需要加载权重的层的 id。CnnlNet 中将创建的层存储在一个数组中, id 即为当前层在该数组中的下标, 比如第一个层的 id 为 0, 第二个层的 id 则为 1 filter_data (list): 权重数据。必须是一维数组 bias_data(list): 参数偏置
setInputData: pycnnl.CnnlNet().setInputData(input_data)	加载输入数据	input_data (list): 输入数据。必须是一维数组, 数据布局为 NHWC
forward: pycnnl.CnnlNet().forward()	进行前向传播计算	
getOutputData: pycnnl.CnnlNet().getOutputData()	获取网络的计算结果	返回值 output_data: 网络最后一层的计算结果
size: pycnnl.CnnlNet().size()	获取当前神经网络中层的数量	返回值 layers_num (int): 当前网络中层的数量

下面以代码示例2.10为例，介绍如何调用 pycnnl 提供的编程接口来创建网络层。首先

实例化 `pycnnl.CnnlNet()`，然后调用 `CnnlNet` 中的 `createXXXLayer` 成员函数就可以创建相应的网络层，例如创建全连接层时只需调用 `pycnnl.CnnlNet().createMlpLayer`。所有创建好的层对象的指针会按顺序以数组的形式保存在 `CnnlNet` 中，数组的下标作为层的 id 使用，当调用 `pycnnl.CnnlNet().loadParams` 函数时，便可以通过此 id 来指定需要加载参数的层。`pycnnl.CnnlNet().forward` 函数会遍历层数组中的对象，依次调用每个网络层的前向传播函数，最终返回最后一层的前向传播结果。

代码示例 2.10 pycnnl 创建层程序示例

```

1 # 实例化 CnnlNet
2 net = pycnnl.CnnlNet()
3 # 设定神经网络输入维度
4 net.setInputShape(1, 3, 224, 224)
5 # conv1_1
6 # 创建卷积层
7 input_shape1 = pycnnl.IntVector(4)
8 input_shape1[0] = 1
9 input_shape1[1] = 3
10 input_shape1[2] = 224
11 input_shape1[3] = 224
12 net.createConvLayer('conv1_1', input_shape1, 64, 3, 1, 1, 1)
13 # relu1_1
14 net.createReLuLayer('relu1_1')

```

在使用 `pycnnl` 之前，需要安装 `pycnnl` 库：先解压 `pycnnl.tar.gz`，再进入 `pycnnl` 目录，执行 `build_pycnnl.sh` 脚本进行编译和安装。安装完成后，进入 `pycnnl/env` 目录下，执行 `source env.sh` 命令，之后便可以在 Python 程序中调用 `pycnnl` 库。调用 DLP 的 `pycnnl` 库的 Python 程序，编译运行方式与 CPU 上的方式一致。

感兴趣的同学可以进一步阅读 `pycnnl` 源码中网络层的实现，了解如何调用 `CNNL` 库中的高性能算子实现全连接层的基本功能，ReLU 层和 Softmax 层的底层实现与之类似，具体每一层的 C++ 代码可以在 `pycnnl/src/layers` 中查看。

2.2.3 实验环境

硬件环境：DLP。

软件环境：`pycnnl` 库，Python 编译环境及相关的扩展库，包括 Python 3.6.12、Pillow 7.2.0、Scipy 1.2.0、NumPy 1.19.5、`CNNL` 高性能算子库，`CNRT` 运行时库，

数据集：MNIST 手写数字库。

模型文件：模型权重参数文件。

2.2.4 实验内容

使用 Python 封装的深度学习编程库 `pycnnl` 搭建一个三层全连接神经网络，利用训练好的模型实现手写数字图像分类，并在 DLP 上正确运行。

与 2.1.4 节类似，本实验的神经网络工程实现大致分为以下 4 个模块：

- 1) 数据加载模块：读取测试数据并进行预处理。
- 2) 基本单元模块：不同网络层的定义，以及前向传播计算等基本功能。

- 3) 网络结构模块：利用基本单元模块搭建完整的网络。
- 4) 网络推断模块 (inference)：使用已有的神经网络模型，对测试数据进行预测。

2.2.5 实验步骤

2.2.5.1 数据加载模块

本实验采用的数据集依然是 MNIST 手写数字库，且数据读取的函数与第2.1.5.1 节的实现相同。因为本实验只完成推断功能，因此只需读取测试数据，对其进行预处理后存储在 NumPy 矩阵中，方便后续推断时快速读取数据。数据加载模块的实现如代码示例2.11所示。

代码示例 2.11 MNIST 子数据集的读取和预处理

```
1 # file: mnist_mlp_demo.py
2 def load_data(self, data_path, label_path):
3     # TODO: 调用函数 load_mnist 读取和预处理 MNIST 中测试数据的图像和标记
4     test_images = _____
5     test_labels = _____
6     self.test_data = np.append(test_images, test_labels, axis=1)
```

2.2.5.2 基本单元模块

pycnnl 库已经将常用网络层的实现用 Python 语言封装起来，因此可以直接调用 pycnnl 中的相关 Python 接口来实现神经网络的基本单元模块。具体调用方式可以参照代码示例2.10。

2.2.5.3 网络结构模块

网络结构模块可以直接使用 pycnnl 封装好的基本单元接口来搭建一个完整的神经网络。在工程实现中，首先用一个类来定义一个神经网络，然后用类的成员函数来定义神经网络的初始化、建立神经网络结构等基本操作。DLP 上实现的网络结构模块如代码示例2.12所示。

代码示例 2.12 三层神经网络的网络结构模块的 DLP 实现

```
1 # file: mnist_mlp_demo.py
2 class MNIST_MLP(object):
3     def __init__(self):
4         # 初始化网络，创建 pycnnl.CnnlNet() 实例
5         self.net = pycnnl.CnnlNet()
6
7
8     def build_model(self, batch_size=100, input_size=784,
9                    hidden1=32, hidden2=16, out_classes=10):
10        # 使用 pycnnl 的接口建立三层神经网络结构
11        self.batch_size = batch_size
12        self.out_classes = out_classes
13
14        # 创建神经网络的层
15        self.net.setInputShape(batch_size, input_size, 1, 1)
16        # TODO: 使用 pycnnl 搭建三层神经网络结构
17        # fcl
```



```

18         input_shapem1 = pycnnl.IntVector(4)
19         input_shapem1[0] = batch_size
20         input_shapem1[1] = 1
21         input_shapem1[2] = 1
22         input_shapem1[3] = input_size
23         weight_shapem1 = pycnnl.IntVector(4)
24         weight_shapem1[0] = batch_size
25         weight_shapem1[1] = 1
26         weight_shapem1[2] = input_size
27         weight_shapem1[3] = hidden1
28         output_shapem1 = pycnnl.IntVector(4)
29         output_shapem1[0] = batch_size
30         output_shapem1[1] = 1
31         output_shapem1[2] = 1
32         output_shapem1[3] = hidden1
33         self.net.createMlpLayer('fc1', hidden1)
34         -----

```

在该示例程序中定义了以下成员函数：

- 网络初始化：创建 `pycnnl.CnnlNet()` 的实例 `net`。后续神经网络层的创建、参数的加载、前向传播计算等操作都通过该对象来调用。
- 建立网络结构：首先加载输入数据和模型权重的参数文件，然后定义整个神经网络的拓扑结构。定义网络结构时，使用 `net` 中的 `createXXXLayer` 函数来实例化每一层。

2.2.5.4 网络推断模块

搭建好网络后，就可以加载训练好的模型、输入数据进行预测。网络推断模块的 DLP 实现如代码示例2.13所示。

代码示例 2.13 三层神经网络的网络推断模块的 DLP 实现示例

```

1 # file: mnist_mlp_demo.py
2 def load_model(self, param_dir):
3     # TODO: 分别为三层全连接层加载参数
4     params = np.load(param_dir, allow_pickle=True, encoding="latin1").item()
5     weigh1 = params['w1'].flatten().astype(np.float)
6     bias1 = params['b1'].flatten().astype(np.float)
7     self.net.loadParams(0, weigh1, bias1)
8     weigh2 = params['w2'].flatten().astype(np.float)
9     bias2 = params['b2'].flatten().astype(np.float)
10    -----
11    weigh3 = params['w3'].flatten().astype(np.float)
12    bias3 = params['b3'].flatten().astype(np.float)
13    -----
14
15 def forward(self): # 前向传播
16     return self.net.forward()
17
18 def evaluate(self):
19     pred_results = np.zeros([self.test_data.shape[0]])
20     # 读取一定批量的测试数据进行前向传播
21     for idx in range(self.test_data.shape[0]//self.batch_size):
22         batch_images = self.test_data[idx*self.batch_size:(idx+1)*self.batch_size, :-1]
23         data = batch_images.flatten().tolist()
24         # 加载输入数据
25         self.net.setInputData(data)
26         # 打印推断的时间
27         start = time.time()

```

```

28     self.forward()
29     end = time.time()
30     print('inferencing time: %f'%(end - start))
31     prob = self.net.getOutputData()
32     prob = np.array(prob).reshape((self.batch_size, self.out_classes))
33     pred_labels = np.argmax(prob, axis=1)
34     pred_results[idx*self.batch_size:(idx+1)*self.batch_size] = pred_labels
35     accuracy = np.mean(pred_results == self.test_data[:, -1])
36     print('Accuracy in test set: %f' % accuracy)

```

在该示例程序中，神经网络推断模块的参数加载、前向传播、精度计算等基本操作拆分为神经网络类的成员函数来定义：

- 神经网络的参数加载：读取模型参数文件，并使用 `net` 中的 `loadParams` 接口加载参数。可以使用第2.1节实验中训练得到的模型参数用于本实验，需要做如下处理：由于 Python 中的浮点数类型 `float` 是双精度浮点，`pycnnl` 接口内部实现的 C++ 函数接收的权重也只能是双精度浮点数类型，而 `numpy` 存储的数据包括权重都是 `np.float32` 类型，因此需要手动将 `numpy` 数据类型转为 `np.float64` 类型，否则在调用 `pycnnl` 库的接口过程中会报错。

- 神经网络的前向传播：`net.forward` 函数会自动遍历调用 `net` 中的每一层的前向传播函数，并将最后一层前向传播的计算结果返回。

- 神经网络推断函数主体：与第2.1节中的 CPU 实现类似，循环读取一定批量的测试数据，随后调用网络的前向传播函数计算得到神经网络的输出结果，然后与测试数据集的标记进行比对计算得到模型的精度。

2.2.5.5 完整实验流程

完成所有模块的实现后，就可以调用上述模块中的函数，在 DLP 上运行神经网络实现手写数字图像分类。网络运行的流程与 CPU 上的执行流程基本一致。本实验中三层神经网络的完整流程的程序示例如代码示例2.14所示。首先实例化三层神经网络对应的类；其次调用网络结构模块 `build_model` 建立神经网络，指定神经网络的超参数（如每层的神经元个数）；随后调用 `load_data` 函数进行数据的加载和预处理；然后调用 `load_model` 函数从文件中读取训练好的模型参数；最后调用 `evaluate` 函数执行网络推断模块得到预测结果，并测试模型的精度。在上一节的 CPU 实验中，我们设置的默认 `batch size` 为 100。对于这种小规模运算，使用 DLP 这样算力强大的设备实在是有些大材小用了，因此我们将 `batch size` 改为 10000，一次传入 10000 张图片，记录 DLP 计算的时间。因为 CNL 在第一次运行的时候会有一个指令生成的过程，导致运行时间会长一些，所以我们多次执行 `evaluate` 函数，排除第一次计算的时间，其它的每次计算时间就和真实的硬件时间很接近了。

用 `load_data` 和 `load_model` 加载数据和参数，CNL 和 CNRT 会在 DLP 上分配内存空间，将参数拷入 DLP 的内存

代码示例 2.14 三层神经网络的完整流程 DLP 实现示例

```

1 # file: mnist_mlp_demo.py
2 if __name__ == '__main__':
3     # 设置 batch_size 为 10000
4     batch_size = 10000
5     h1, h2, c = 32, 16, 10

```

```

6 mlp = MNIST_MLP()
7 mlp.build_model(batch_size=batch_size, hidden1=h1, hidden2=h2, out_classes=c)
8 model_path = '../data/mnist_mlp_data/mlp-%d-%d-10epoch.npy'%(h1,h2)
9 test_data = '../data/mnist_mlp_data/mnist_data/t10k-images-idx3-ubyte'
10 test_label = '../data/mnist_mlp_data/mnist_data/t10k-labels-idx1-ubyte'
11 mlp.load_data(test_data, test_label)
12 mlp.load_model(model_path)
13 # 循环多次统计DLP计算时间
14 for i in range(3):
15     mlp.evaluate()

```

2.2.5.6 实验运行

根据第2.2.5.1节 ~ 第2.2.5.5节的描述补全 mnist_mlp_demo.py 代码, 并通过 Python 运行.py 代码。具体可以参考以下步骤:

1) 环境申请

申请实验环境并登录云平台, 云平台上/opt/code_chap_2_3/code_chap_2_3_student/ 目录下是本实验的示例代码。安装 swig 并加载 pycnnl。

```

# 登录云平台
ssh root@xxx.xxx.xxx -p xxxxx
# 进入 code_chap_2_3_student 目录
cd /opt/code_chap_2_3/code_chap_2_3_student
# 进入 pycnnl/cnnl_python 目录下
cd pycnnl/cnnl_python
# 安装 swig
pip install swig
# 运行 build_pycnnl.sh 脚本文件, 生成扩展模块
bash build_pycnnl.sh

```

2) 代码实现

补全 stu_upload 中的 layers_1.py, mnist_mlp_cpu.py, mnist_mlp_demo.py 文件。

```

# 进入实验目录
cd exp_2_2_mnist_mlp_dlp
# 补全实验代码
vim stu_upload/layers_1.py
vim stu_upload/mnist_mlp_cpu.py
vim stu_upload/mnist_mlp_demo.py

```

其中, 将 mnist_mlp_cpu.py 文件中的 build_mnist_mlp() 函数修改为以下内容:

```

def build_mnist_mlp(param_dir='weight.npy'):
    h1, h2, e = 32, 16, 10
    mlp = MNIST_MLP(hidden1=h1, hidden2=h2, max_epoch=e)
    mlp.load_data()
    mlp.build_model()
    mlp.init_model()
    # mlp.train()
    # mlp.save_model('mlp-%d-%d-%depochn.npy' % (h1, h2, e))
    mlp.load_model(param_dir)
    return mlp

```

3) 运行实验

将实验 2.1 中训练得到的参数复制到 stu_upload 目录下, 并将参数文件重命名为 weight.npy。然后运行实验:

```
# 运行完整实验
python main_exp_2_2.py
```

2.2.6 实验评估

本实验中，精度评判标准与第2.1节实验一样，使用测试集的平均分类正确率判断分类结果的精度。性能评判标准为设置 batch size 为 10000 时，进行一次 forward 的时间。本实验的评分标准设定如下：

- 60 分标准：完善本节实验代码，用 pycnn1 搭建出的三层神经网络能够在 DLP 上进行推断，并且在测试集上的平均分类正确率高于 90%。
- 80 分标准：修改网络的规模，使用第2.1节实验的代码重新训练模型，使训练得到的模型在 DLP 上运行的推断（forward）耗时为 CPU 推断耗时的 1/50 或更低，并且在测试集上的平均分类正确率高于 95%
- 100 分标准：基于三层神经网络重新设计自己的神经网络结构，使用第2.1节实验的代码重新训练模型，使训练得到的模型在 DLP 上运行的推断耗时为 CPU 推断耗时的 1/200 或更低，并且在测试集上的平均分类正确率高于 98%。

2.2.7 实验思考

在实验中请思考如下问题：

- 1) DLP 在运行神经网络推断时，相对于 CPU 有什么优势和劣势？
- 2) 在什么样的神经网络结构下，DLP 能够最大化发挥它的性能优势？